Training OTU ML Model

Cameron DeBellefeuille

2026-01-12

```
# --- SETUP AND LIBRARIES ---
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(tidyr)
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
library(MASS)
```

```
##
## Attaching package: 'MASS'
```

```
## The following object is masked from 'package:dplyr':
##
##     select
```

```
library(randomForest)
```

```
## randomForest 4.7-1.2
```

```
## Type rfNews() to see new features/changes/bug fixes.
```

```
##
## Attaching package: 'randomForest'
```

```
## The following object is masked from 'package:ggplot2':
##
##     margin
```

```
## The following object is masked from 'package:dplyr':
##
##     combine
```

```
library(ggplot2)
library(gbm)
```

```
## Loaded gbm 2.2.2
```

```
## This version of gbm is no longer under development. Consider transitioning to gbm3, https://github.com/gbm-developers/gbm
3
```

```
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
## Attaching package: 'pROC'
```

```
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```
library(gridExtra)
```

```
##
## Attaching package: 'gridExtra'
```

```
## The following object is masked from 'package:randomForest':
##
##     combine
```

```
## The following object is masked from 'package:dplyr':
##
##     combine
```

```
theme_set(theme_classic())
```

```
# --- DATA LOADING ---
# Load the main OTU table
df <- read.csv("C:/Users/21cd51/OneDrive - Queen's University/ASD_ML_proj/otu_table_with_labels_header.csv")

# Ensure metadata columns are factors
df$sample_id <- as.factor(df$sample_id)
df$diagnosis <- as.factor(df$diagnosis)
df$household <- as.factor(df$household)

# Reorder columns for visibility
df <- df %>% dplyr::select(sample_id, diagnosis, household, everything())

# --- PHASE 1: DATA RESHAPING (Linking Mothers to Children) ---

# 1. Filter for Mothers (Specifically ASDM or TDM labels)
mothers <- df %>% filter(diagnosis == "ASDM" | diagnosis == "TDM")

# 2. Filter for Children (Specifically ASD or TD labels)
children <- df %>% filter(diagnosis == "ASD" | diagnosis == "TD")

# 3. Create a Target map from the children's diagnoses
child_status <- children %>%
  dplyr::select(household, child_outcome = diagnosis)

# 4. Join Child Outcome to the Maternal OTU features using Household ID
ml_df <- mothers %>%
  inner_join(child_status, by = "household") %>%
  dplyr::select(-sample_id, -diagnosis, -household)

# 5. Clean up "ghost" factor levels and move target to the front
ml_df$child_outcome <- droplevels(ml_df$child_outcome)
ml_df <- ml_df %>% relocate(child_outcome)
```

```
# --- PHASE 2: MACHINE LEARNING PIPELINE ---

# Step 5: Convert OTU features to numeric
ml_df[, names(ml_df) != "child_outcome"] <- lapply(ml_df[, names(ml_df) != "child_outcome"],
                                                   function(x) as.numeric(as.character(x)))

# Step 6: Sparsity Filtering (Keep OTUs present in >= 10% of samples)
presence_threshold <- 0.1 * nrow(ml_df)
ml_df <- ml_df[, c(TRUE, colSums(ml_df[,-1] > 0) > presence_threshold)]

# Step 7: Log Transformation (log1p)
ml_df[,-1] <- log1p(ml_df[,-1])

# Step 8: Split data (80% Train, 20% Test)
set.seed(123)
train_index <- createDataPartition(ml_df$child_outcome, p = 0.8, list = FALSE)
train_data <- ml_df[train_index, ]
test_data  <- ml_df[-train_index, ]

# Step 9: Training the Gradient Boosting Model (GBM)
set.seed(123)
train_data$y <- ifelse(train_data$child_outcome == "ASD", 1, 0) # Convert to binary for bernoulli

model_boost <- gbm(y ~ . -child_outcome,
                   data = train_data,
                   distribution = "bernoulli",
                   n.trees = 500,
                   interaction.depth = 4,
                   shrinkage = 0.01,
                   cv.folds = 5)

# Step 10: Evaluation on Test Data
probs <- predict(model_boost, test_data, n.trees = 500, type = "response")
preds <- ifelse(probs > 0.5, "ASD", "TD")

# Confusion Matrix
confusionMatrix(as.factor(preds),test_data$child_outcome)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction ASD TD
##        ASD  15  1
##        TD    0  8
##
##                Accuracy : 0.9583
##                  95% CI : (0.7888, 0.9989)
##     No Information Rate : 0.625
##     P-Value [Acc > NIR] : 0.0001944
##
##                   Kappa : 0.9091
##
##  Mcnemar's Test P-Value : 1.0000000
##
##             Sensitivity : 1.0000
##             Specificity : 0.8889
##          Pos Pred Value : 0.9375
##          Neg Pred Value : 1.0000
##              Prevalence : 0.6250
##          Detection Rate : 0.6250
##    Detection Prevalence : 0.6667
##       Balanced Accuracy : 0.9444
##
##        'Positive' Class : ASD
##
```

```
# Step 11 Results and Visualization
# Load the taxonomy file
# sep = "\t" specifies it is a tab-separated text file
# header = FALSE because Greengenes files don't usually have column titles
tax_master <- read.table("97_otu_taxonomy.txt",
                         header = FALSE,
                         sep = "\t",
                         stringsAsFactors = FALSE)

# Assign clear column names
colnames(tax_master) <- c("otu", "Taxonomy")

# Important: Add the "X" prefix to match the format R gave your OTU table columns
tax_master$otu <- paste0("X", tax_master$otu)

# Verify the first few rows
head(tax_master)
```

```
##       otu
## 1 X367523
## 2 X187144
## 3 X836974
## 4 X310669
## 5 X823916
## 6 X878161
##                                                                                                                                Taxonomy
## 1 k__Bacteria; p__Bacteroidetes; c__Flavobacteriia; o__Flavobacteriales; f__Flavobacteriaceae; g__Flavobacterium; s__
## 2                                          k__Bacteria; p__Firmicutes; c__Clostridia; o__Clostridiales; f__; g__; s__
## 3                                           k__Bacteria; p__Cyanobacteria; c__Chloroplast; o__Cercozoa; f__; g__; s__
## 4                                          k__Bacteria; p__Firmicutes; c__Clostridia; o__Clostridiales; f__; g__; s__
## 5 k__Bacteria; p__Proteobacteria; c__Gammaproteobacteria; o__Pseudomonadales; f__Moraxellaceae; g__Enhydrobacter; s__
## 6    k__Bacteria; p__Acidobacteria; c__Acidobacteriia; o__Acidobacteriales; f__Acidobacteriaceae; g__Terriglobus; s__
```

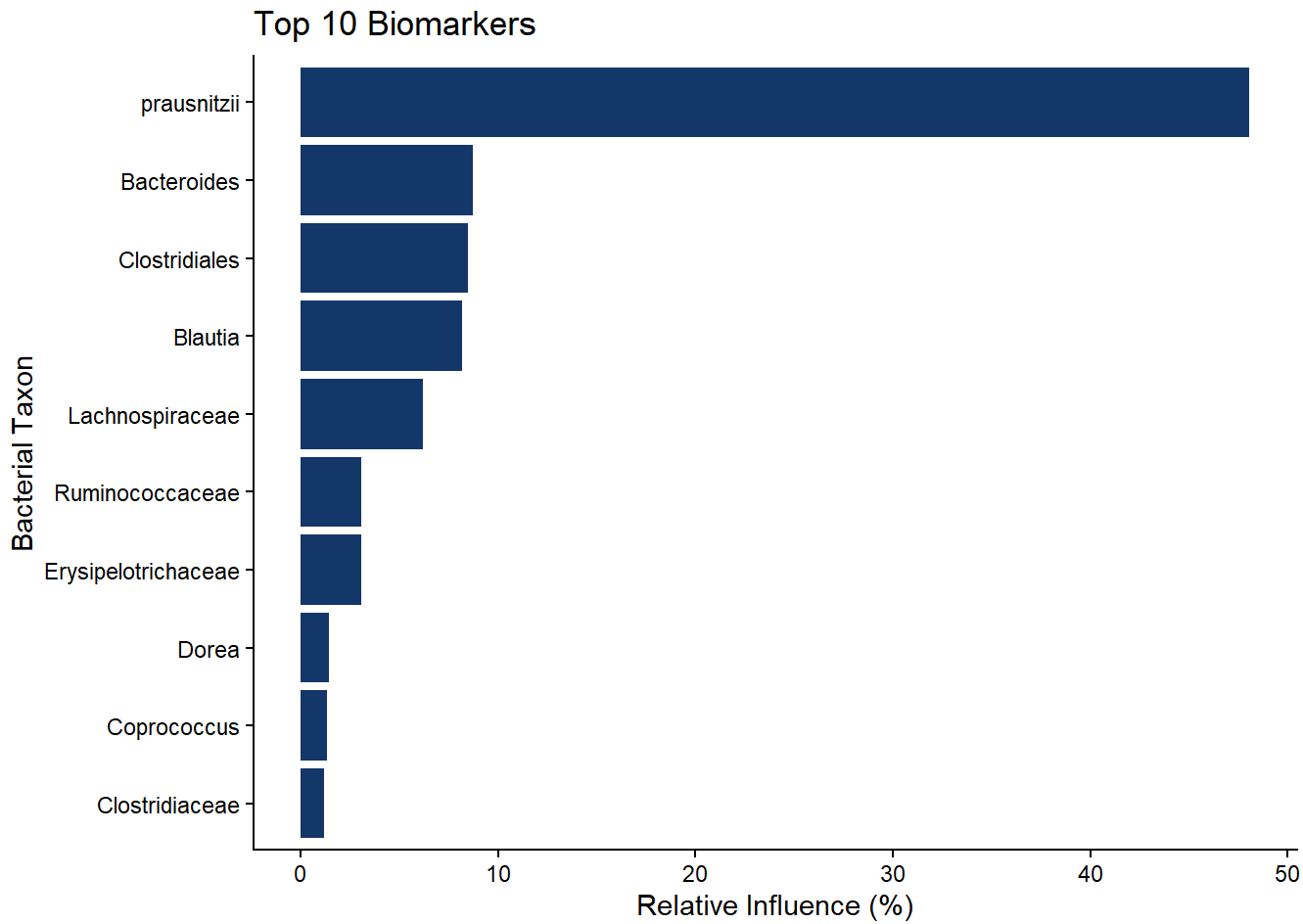
```
# Get model influence
otu_inf <- summary(model_boost, plotit = FALSE)
colnames(otu_inf) <- c("otu", "rel.inf")

# Merge names from tax_master into your results
otu_inf <- merge(otu_inf, tax_master, by = "otu", all.x = TRUE)

#test <- otu_inf1 %>% inner_join(tax_master, by = "otu")

head(otu_inf)
```

```
##       otu     rel.inf
## 1 X100198 0.052235509
## 2 X102017 0.004349559
## 3 X102159 0.000000000
## 4 X102221 0.060469581
## 5 X104394 0.000000000
## 6 X104551 0.000000000
##                                                                                                Taxonomy
## 1                  k__Bacteria; p__Firmicutes; c__Clostridia; o__Clostridiales; f__Lachnospiraceae; g__; s__
## 2 k__Bacteria; p__Firmicutes; c__Clostridia; o__Clostridiales; f__Lachnospiraceae; g__Pseudobutyrivibrio; s__
## 3        k__Bacteria; p__Firmicutes; c__Clostridia; o__Clostridiales; f__Lachnospiraceae; g__Coprococcus; s__
## 4                  k__Bacteria; p__Firmicutes; c__Clostridia; o__Clostridiales; f__Lachnospiraceae; g__; s__
## 5                  k__Bacteria; p__Firmicutes; c__Clostridia; o__Clostridiales; f__Lachnospiraceae; g__; s__
## 6                  k__Bacteria; p__Firmicutes; c__Clostridia; o__Clostridiales; f__Lachnospiraceae; g__; s__
```

```
# Create a 'Clean_Label' column
# This defaults to the OTU ID if no taxonomy is found
otu_inf$Clean_Label <- as.character(otu_inf$otu)

# Pre-clean the strings: Remove the empty trailing levels like "g__; s__"
# This "strips" the empty markers so the deepest level is a real name
otu_inf$Cleaned_Tax <- gsub("[a-z]__; ", "", otu_inf$Taxonomy)
otu_inf$Cleaned_Tax <- gsub("[a-z]__$", "", otu_inf$Cleaned_Tax)

# If taxonomy exists, strip the messy k__, p__, etc. to get the most specific name
has_tax <- !is.na(otu_inf$Cleaned_Tax) & otu_inf$Cleaned_Tax != ""
otu_inf$Clean_Label[has_tax] <- sub(".*[a-z]__", "", otu_inf$Cleaned_Tax[has_tax])

# Final touch: Remove trailing semicolons and extra spaces
otu_inf$Clean_Label <- trimws(gsub(";", "", otu_inf$Clean_Label))

head(otu_inf)
```

```
##       otu     rel.inf
## 1 X100198 0.052235509
## 2 X102017 0.004349559
## 3 X102159 0.000000000
## 4 X102221 0.060469581
## 5 X104394 0.000000000
## 6 X104551 0.000000000
##                                                                                  Taxonomy
## 1                  k__Bacteria; p__Firmicutes; c__Clostridia; o__Clostridiales; f__Lachnospiraceae; g__; s__
## 2 k__Bacteria; p__Firmicutes; c__Clostridia; o__Clostridiales; f__Lachnospiraceae; g__Pseudobutyrivibrio; s__
## 3        k__Bacteria; p__Firmicutes; c__Clostridia; o__Clostridiales; f__Lachnospiraceae; g__Coprococcus; s__
## 4                  k__Bacteria; p__Firmicutes; c__Clostridia; o__Clostridiales; f__Lachnospiraceae; g__; s__
## 5                  k__Bacteria; p__Firmicutes; c__Clostridia; o__Clostridiales; f__Lachnospiraceae; g__; s__
## 6                  k__Bacteria; p__Firmicutes; c__Clostridia; o__Clostridiales; f__Lachnospiraceae; g__; s__
##         Clean_Label
## 1    Lachnospiraceae
## 2 Pseudobutyrivibrio
## 3       Coprococcus
## 4    Lachnospiraceae
## 5    Lachnospiraceae
## 6    Lachnospiraceae
##                                                                                      Cleaned_Tax
## 1                       k__Bacteria; p__Firmicutes; c__Clostridia; o__Clostridiales; f__Lachnospiraceae;
## 2 k__Bacteria; p__Firmicutes; c__Clostridia; o__Clostridiales; f__Lachnospiraceae; g__Pseudobutyrivibrio;
## 3        k__Bacteria; p__Firmicutes; c__Clostridia; o__Clostridiales; f__Lachnospiraceae; g__Coprococcus;
## 4                       k__Bacteria; p__Firmicutes; c__Clostridia; o__Clostridiales; f__Lachnospiraceae;
## 5                       k__Bacteria; p__Firmicutes; c__Clostridia; o__Clostridiales; f__Lachnospiraceae;
## 6                       k__Bacteria; p__Firmicutes; c__Clostridia; o__Clostridiales; f__Lachnospiraceae;
```

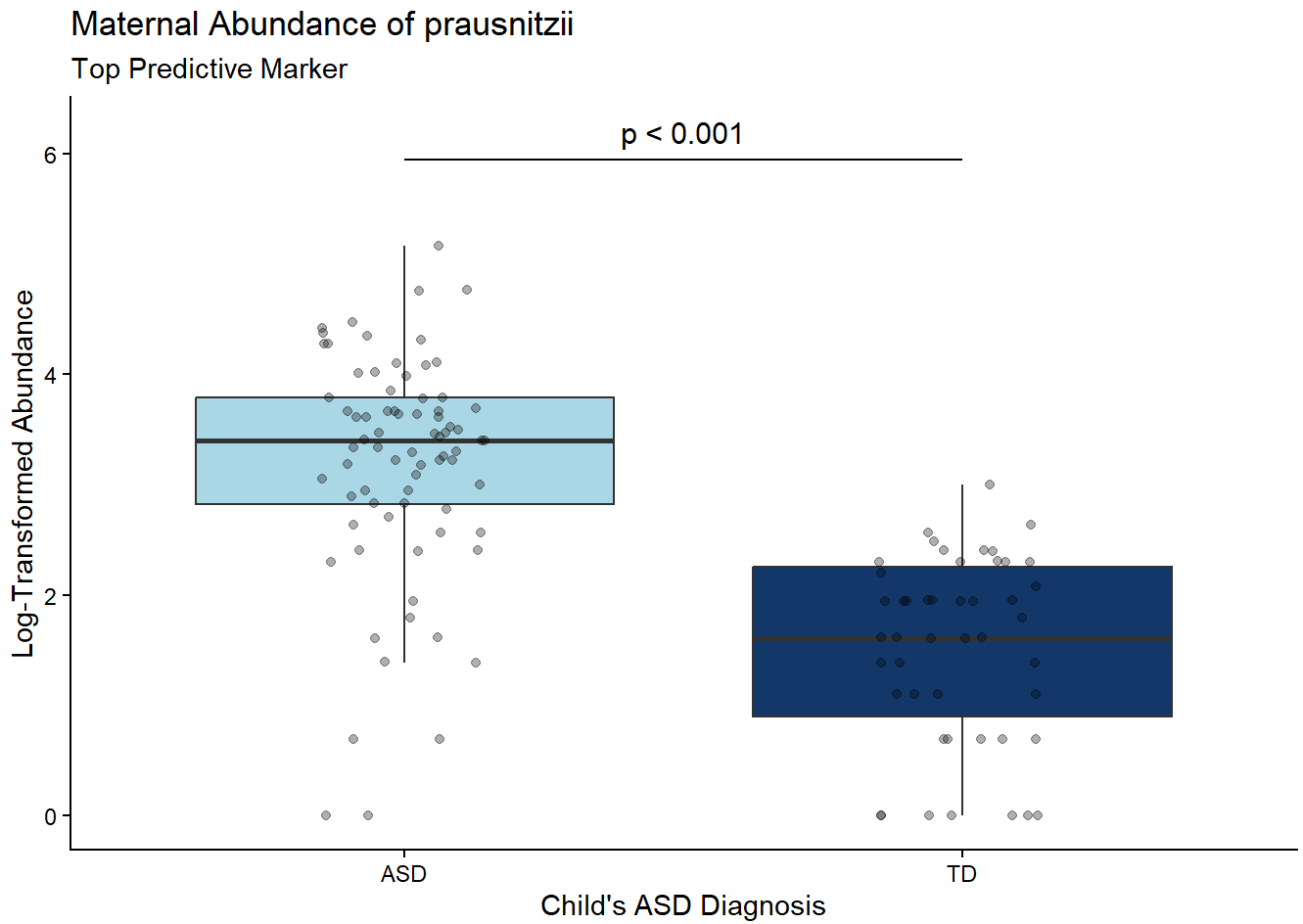
```
plot_data <- otu_inf %>%
  group_by(Clean_Label) %>%
  summarise(rel.inf = sum(rel.inf)) %>%
  slice_max(rel.inf, n = 10)

plot_top10 <- ggplot(plot_data,
       aes(x = rel.inf, y = reorder(Clean_Label, rel.inf))) +
  geom_col(fill = "#143a6d") +
  theme_classic() +
  labs(title = "Top 10 Biomarkers",
       x = "Relative Influence (%)",
       y = "Bacterial Taxon")
plot_top10
```




```
# Step 12 & 13: Confusion Matrix & Metrics
CatDat <- data.frame(
  Obs = test_data$child_outcome,
  Pred = factor(ifelse(probs > 0.5, "ASD", "TD"), levels = levels(test_data$child_outcome))
)

misclassification_rate <- sum(CatDat$Obs != CatDat$Pred) / nrow(CatDat)
cat("Misclassification Rate:", misclassification_rate, "\n")
```

```
## Misclassification Rate: 0.04166667
```

```
# Step 17: Boxplot of Top Influential Microbe
# NOW grab the ID and Name from the top row
otu_inf <- otu_inf[order(-otu_inf$rel.inf), ]
top_otu_id   <- as.character(otu_inf$otu[1])
top_otu_name <- as.character(otu_inf$Clean_Label[1])

# Create the plot
# 1. Run the t-test to get the exact p-value
t_test_res <- t.test(as.formula(paste0("`", top_otu_id, "` ~ child_outcome")), data = ml_df)
p_val <- t_test_res$p.value

# 2. Format the p-value for the plot
# This shows "p < 0.001" if very small, or the exact number if larger
p_label <- if(p_val < 0.001) "p < 0.001" else paste0("p = ", round(p_val, 4))

# 3. Find the highest point in the data to place the bar above it
y_max <- max(ml_df[[top_otu_id]], na.rm = TRUE)
y_pos <- y_max * 1.15  # Position the label 15% above the highest data point

# 4. Create the plot using only base ggplot2
plot_abundance1 <- ggplot(ml_df, aes(x = child_outcome, y = .data[[top_otu_id]], fill = child_outcome)) +
  geom_boxplot(outlier.shape = NA) +
  geom_jitter(width = 0.15, alpha = 0.3) +
  # Adding the manual significance line (the "bracket")
  annotate("segment", x = 1, xend = 2, y = y_pos, yend = y_pos, colour = "black") +
  # Adding the p-value text above the line
  annotate("text", x = 1.5, y = y_pos + (y_max * 0.05), label = p_label, size = 4) +
  labs(title = paste("Maternal Abundance of", top_otu_name),
       subtitle = paste("Top Predictive Marker"),
       y = "Log-Transformed Abundance",
       x = "Child's ASD Diagnosis") +
  theme_classic() +
  scale_fill_manual(values = c("ASD" = "lightblue", "TD" = "#143a6d")) +
  theme(legend.position = "none")
plot_abundance1
```

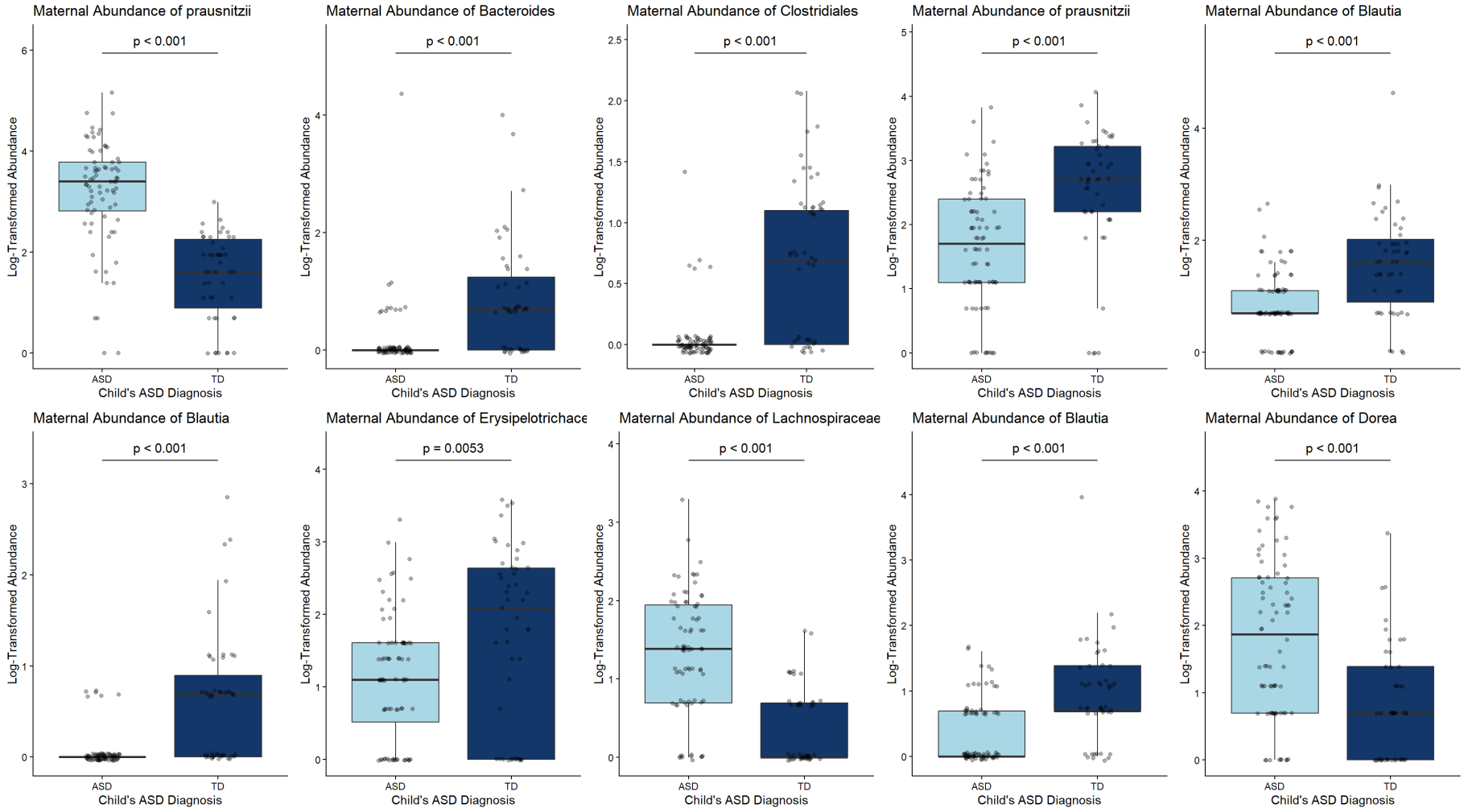



```
otu_inf_summary <- otu_inf[order(-otu_inf$rel.inf), ]
top_10_otus <- as.character((otu_inf$otu))
```

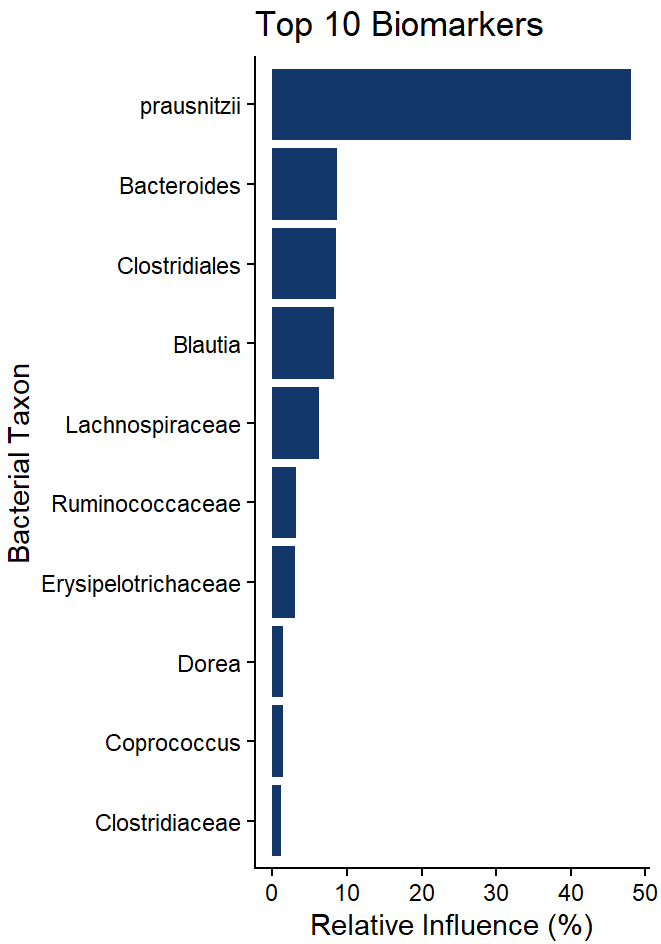
```
p_results <- lapply(top_10_otus, function(otu) {
  formula_obj <- as.formula(paste0("`", otu, "` ~ child_outcome"))
  t_res <- t.test(formula_obj, data = ml_df)

  data.frame(
    OTU = otu,
    p_value = t_res$p.value
  )
})

p_results_df <- do.call(rbind, p_results)
p_results_df$p_adj <- p.adjust(p_results_df$p_value, method = "BH")
```

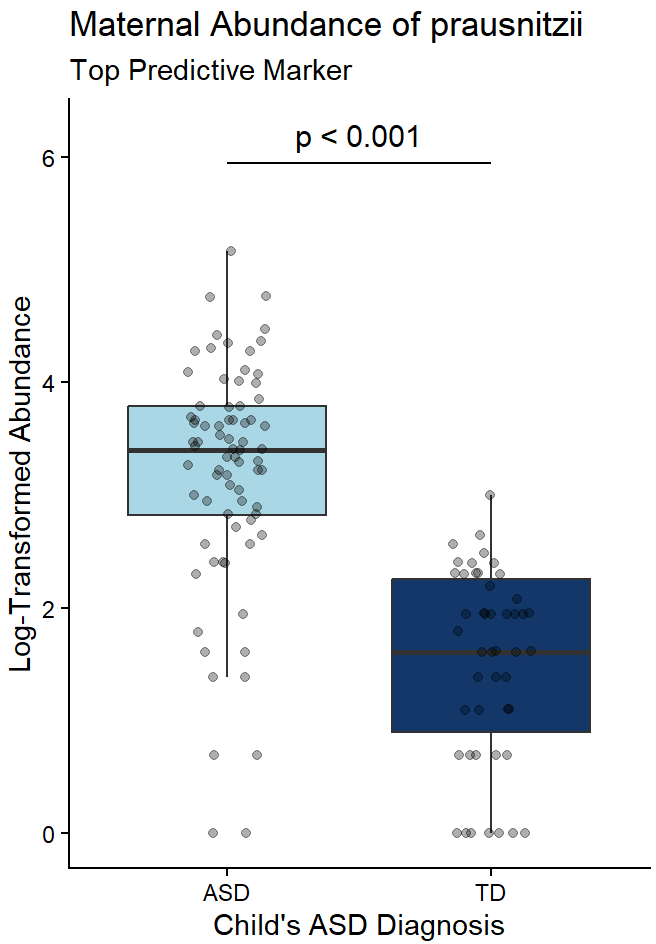
```
library(gridExtra)

# Step 17: Boxplots of Top 10 Influential Microbes

# FIRST: Re-sort by influence
otu_inf <- otu_inf[order(-otu_inf$rel.inf), ]

# Create empty list to store plots
plot_list <- list()

# Loop through top 10 OTUs
for(i in 1:10) {

  # NOW grab the ID and Name from the current row
  top_otu_id   <- as.character(otu_inf$otu[i])
  top_otu_name <- as.character(otu_inf$Clean_Label[i])

  # 1. Run the t-test to get the exact p-value
  t_test_res <- t.test(as.formula(paste0("`", top_otu_id, "` ~ child_outcome")),
                       data = ml_df)
  p_val <- t_test_res$p.value

  # 2. Format the p-value for the plot
  p_label <- if(p_val < 0.001) "p < 0.001" else paste0("p = ", round(p_val, 4))

  # 3. Find the highest point in the data to place the bar above it
  y_max <- max(ml_df[[top_otu_id]], na.rm = TRUE)
  y_pos <- y_max * 1.15

  # 4. Create the plot using only base ggplot2
  plot_abundance <- ggplot(ml_df,
                           aes(x = child_outcome,
                               y = .data[[top_otu_id]],
                               fill = child_outcome)) +
    geom_boxplot(outlier.shape = NA) +
    geom_jitter(width = 0.15, alpha = 0.3) +

    # Adding the manual significance line (the "bracket")
    annotate("segment", x = 1, xend = 2,
```

```
             y = y_pos, yend = y_pos, colour = "black") +

    # Adding the p-value text above the line
    annotate("text", x = 1.5,
             y = y_pos + (y_max * 0.05),
             label = p_label, size = 4) +

    labs(title = paste("Maternal Abundance of", top_otu_name),
         y = "Log-Transformed Abundance",
         x = "Child's ASD Diagnosis") +

    theme_classic() +
    scale_fill_manual(values = c("ASD" = "lightblue",
                                 "TD" = "#143a6d")) +
    theme(legend.position = "none")

  # Store plot in list
  plot_list[[i]] <- plot_abundance
}

# Combine all 10 plots
grid.arrange(grobs = plot_list, ncol = 5)
```




```
grid.arrange(plot_top10, plot_abundance1, ncol = 2)
```


Top 10 Biomarkers
prausnitzii
Bacteroides
Clostridiales
Blautia
Lachnospiraceae
Ruminococcaceae
Erysipelotrichaceae
Dorea
Coprococcus
Clostridiaceae
Bacterial Taxon
0
10
20
30
40
50
Relative Influence (%)


Maternal Abundance of prausnitzii
Top Predictive Marker
p < 0.001
Log-Transformed Abundance
0
2
4
6
ASD
TD
Child's ASD Diagnosis